

Re:Learn

Adaptive Multimodal Learning Environment — the complete project, explained end to end

“Re:Learn doesn't tell you that you're wrong. It tells you why, fixes that, and checks that the fix stuck.”

An AI tutor for introductory Newtonian mechanics that reads a learner's answer and explanation together, names the misconception behind the response, delivers a targeted intervention with a physics animation, and verifies the fix with transfer questions in new contexts — resolved only when the answer is right **and** the reasoning is clean, in two contexts, plus a delayed recheck.

Version	1.0 — built against Re:Learn PRD v1.0 (3 October 2026)
Stack	Python 3.10+, FastAPI, scikit-learn, SQLite, vanilla HTML/CSS/JS + canvas
Run	python run.py (first run: dataset → train → evaluate → serve, ~10 s on CPU)
Project folder	relearn/ (backend, frontend, ml, tests, demo, docs)
Author	Prof. Aiman Kazi · Visual Labs

Contents

- 1. The idea in one page
- 2. The learner's journey (with screenshots)
- 3. System architecture and data flow
- 4. Content layer: misconceptions, questions, interventions, probes
- 5. The dataset
- 6. The model and the decision rule
- 7. Resolution logic and the learner model
- 8. Evaluation: does it actually diagnose?
- 9. API reference
- 10. The frontend and the simulations
- 11. File-by-file guide
- 12. How to run (Windows / VS Code)
- 13. Three-minute demo script
- 14. PRD traceability
- 15. Limitations and what comes next
- 16. Version 2: beyond the PRD

1. The idea in one page

The problem

Most learning platforms grade an answer right or wrong and then show the solution. That discards the most useful information a wrong answer carries: **which mistaken idea produced it**. Two students can pick the same wrong option for completely different reasons and need completely different help. Worse, a student can pick the *right* option while holding the wrong idea — and get marked fully correct.

The PRD's own example, which Re:Learn separates on question 1

Student	Picked	Explanation	Re:Learn's diagnosis
A	Acceleration = 0	"It stops at the top, so velocity is zero."	Velocity–acceleration confusion
R	Acceleration = 0	"The force of the throw has run out."	Impetus (motion carries a force)
S	9.8 m/s ² down (correct)	"It is still slowing down because of the throw force."	Right answer, wrong idea → Impetus

A traditional LMS marks A and R identically and marks S as fully correct. Re:Learn gives all three a different, targeted intervention — and only counts an idea as fixed when it stops showing up in the learner's own words.

The loop

Step	Learner	System
1 Answer	Chooses an option, rates confidence (guessing / fairly sure / certain), explains in a sentence	Captures answer + explanation + confidence
2 Diagnose	Sees the verdict with model confidence	Returns correct / flawed_reasoning / misconception / unknown
3 Fix	Reads the explanation, plays the simulation, follows the worked example	Intervention built for that specific misconception
4 Verify	Answers a transfer question in a new context, with reasoning	Resolved only if answer right AND misconception absent, ×2 contexts
5 Track	Watches the thinking map update	active → improving → resolved; recheck later; recurring if it returns
6 Report	Finishes the session	Right first time / found / fixed / still open / recurring

What makes it different

- **Diagnoses the idea, not the answer** — a trained classifier reads the free-text reasoning; the chosen option narrows the candidates, the words decide between them.
- **Finds misconceptions hidden behind correct answers** (flawed-reasoning detection).
- **Abstains rather than guesses** — weak evidence returns *unknown* and asks the learner to elaborate.
- **Verifies learning, not recall** — two clean transfer probes plus a delayed recheck; a correct follow-up alone is never proof.
- **Shows its work** — confidence and candidate probabilities are always visible, and the learner can say "that's not what I meant", which becomes training data.

- **Works for new misconceptions without retraining** — a description-embedding fallback picks up an idea that has a description but no training rows.
- **Proves it** — a full evaluation suite, split by question, served live inside the app.

2. The learner's journey

Everything below is a real session in the shipped app (learner “Aiman”, Quick session, question 1).

Re:Learn Learn Thinking map Teacher Evaluation About

1 Answer 2 Diagnose 3 Fix 4 Verify 5 Track

Re:Learn doesn't tell you that you're wrong. It tells you *why* — and checks that the fix stuck.

Answer a mechanics question, explain your thinking in a sentence. The model names the misconception behind your reasoning, fixes exactly that, and then tests you in a new situation.

Your name

e.g. Aiman

Session

Quick (8 questions) One topic Full bank (30)

Start learning →

1 · Answer 2 · Diagnose 3 · Fix 4 · Verify 5 · Track

Your thinking map
Start a session to begin.

Session

— answered — found — fixed

Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

Start screen: name, session type (Quick 8 / one topic / full 30), and the five-step loop.

Learn

Thinking map

Teacher

Evaluation

About

Aiman

A ball is thrown straight up. At the very top of its flight, what is its acceleration? (Ignore air resistance.)

A Zero — it is momentarily at rest

B 9.8 m/s² downward

C 9.8 m/s² upward

D It depends on how hard it was thrown

Explain your answer in your own words. Speak it — the words matter more than the option.

Tap and speak

Say it in your own words — I'll transcribe it here.

it stops at the top so the velocity is zero

I heard:

velocity

rest

reason

That's enough for me to work with.

Photo of handwriting

or type / edit the transcript above

How sure are you?

guessing

fairly sure

certain

10 words

Diagnose my thinking →

Ctrl+Enter to submit

Your thinking map

Q 1 / 9

Nothing found yet. Explain your reasoning and I'll look for the idea behind it.

Session

0

answered

0

found

0

fixed

Finish & see report

Ask the tutor

Speak a physics question — "what is inertia?", "why does it come back down?"

what is acceleration?

Acceleration is how quickly the velocity is changing ($a = \Delta v / \Delta t$). It is not the velocity itself: something can be momentarily at rest and still be accelerating, like a ball at the top of its flight.

from: glossary

Ask a physics question

Ask

Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

A question: four options; a voice-first explanation panel (tap and speak → transcript → live NLP readout of what was heard); the Ask-the-tutor voice chatbot in the sidebar.

Re:Learn — Adaptive Multimodal Learning Environment · Project explainer

Page 5

1 Answer
2 Diagnose
3 Fix
4 Verify
5 Track

Re:Learn
Learn
Thinking map
Teacher
Evaluation
About
Aiman

The answer is wrong — and I can see why.

model confidence 98% · 28.3 ms · you said: certain

Velocity–acceleration confusion

Zero velocity means zero acceleration

WHAT YOUR REASONING SUGGESTS
If an object is momentarily at rest, nothing is changing, so its acceleration must be zero too. Velocity and acceleration rise and fall together.

WHAT PHYSICS SAYS
Acceleration is the rate at which velocity changes, not the velocity itself. An object can be at rest for a single instant while its velocity is changing rapidly — so the acceleration is non-zero. A ball at the top of its flight has $v = 0$ but $a = 9.8 \text{ m/s}^2$ downward, which is exactly why it starts falling.

How sure is the model?

98% that this is the idea behind your words

Velocity–acceleration confusion 98%

Force causes constant speed 1%

No misconception 1%

Impetus: motion carries a force 0%

► That's not what I meant

Your thinking map

Q 1 / 8

Velocity–acceleration confusion
seen 1x · held with certainty **ACTIVE**

Session

1	1	0
answered	found	fixed

Finish & see report

Model: tldif-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

Velocity says where you are going. Acceleration says how that is changing.

Picture the ball's velocity as an arrow. On the way up the arrow points up and shrinks; at the top it has length zero; on the way down it points down and grows. Something is changing that arrow the whole time — and the thing that changes velocity is acceleration.

Gravity does not switch off at the top. It pulls down at every instant, so the acceleration is 9.8 m/s^2 downward at every instant, including the one where the velocity happens to be zero.

Rule of thumb: zero velocity tells you nothing about acceleration. Ask instead 'is the velocity about to change?' If yes, acceleration is not zero.

$t = 1.22 \text{ s}$

$v = 0$ but $a \neq 0$

$v = +0.5 \text{ m/s}$

$a = -9.8 \text{ m/s}^2$

velocity vs time
slope = $a = \text{constant}$

acceleration vs time

Pause Replay Step ☐ Show what the belief predicts

A ball is thrown straight up. The green arrow is velocity: it shrinks to zero at the top and flips. The amber arrow is acceleration: always 9.8 m/s^2 downward, even at the top. In belief mode a red ghost shows the acceleration arrow wrongly vanishing at the top.

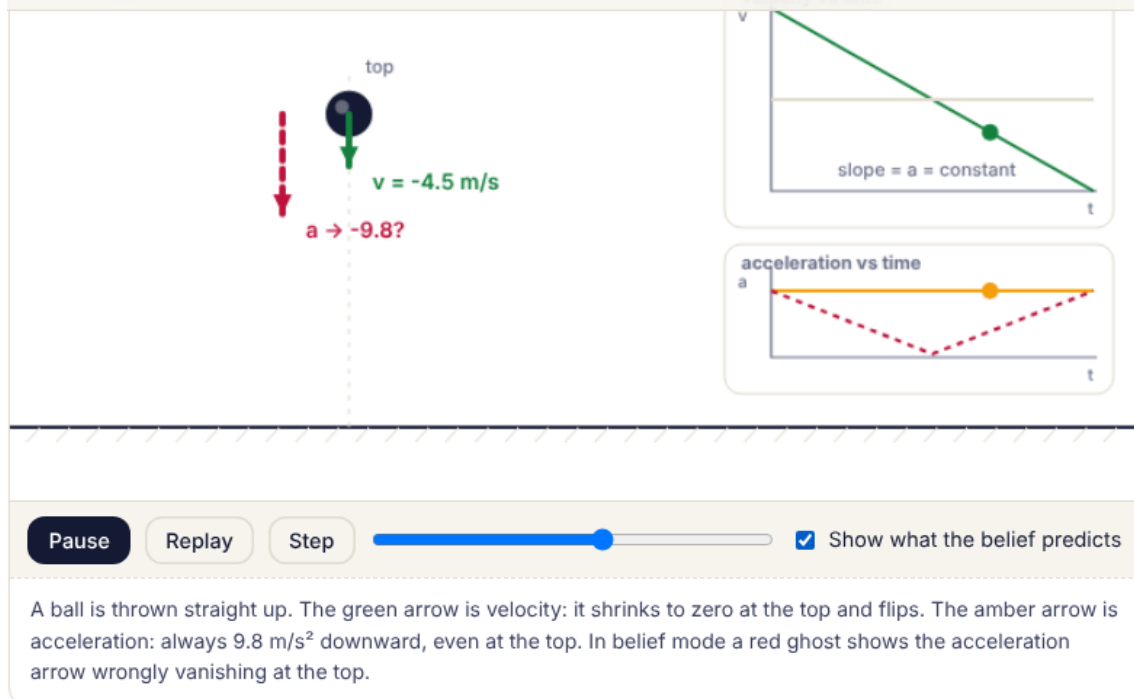
Worked example: A ball thrown up at 10 m/s

- $t = 0 \text{ s}$: $v = +10 \text{ m/s}$ (up). $a = -9.8 \text{ m/s}^2$.
- $t = 0.5 \text{ s}$: $v = 10 - 9.8 \cdot 0.5 = +5.1 \text{ m/s}$. $a = -9.8 \text{ m/s}^2$.
- $t = 1.02 \text{ s}$: $v = 0$ (the top). $a = -9.8 \text{ m/s}^2$ — unchanged.
- $t = 1.5 \text{ s}$: $v = 10 - 9.8 \cdot 1.5 = -4.7 \text{ m/s}$ (down). $a = -9.8 \text{ m/s}^2$.
- The velocity passes through zero; the acceleration never does.

Check yourself: Can something be at rest and accelerating at the same time? (Yes — any turnaround point.)

Test the fix in a new situation → A transfer question — same idea, different context.

Diagnosis + intervention. Top: status banner with model confidence and latency. Left: what the reasoning suggests vs what physics says. Right: confidence meter, candidate probabilities, “that’s not what I meant”. Below: headline, explanation, simulation, worked example, check-yourself.



The simulation in belief mode: the red dashed ghost shows what the misconception predicts (acceleration vanishing at the top) against what actually happens.

Re:Learn

Learn Thinking map Teacher Evaluation About Aiman

A child on a swing reaches the highest point of the swing and pauses for an instant. What is the child's acceleration at that moment?

A Zero, since the child is momentarily still

B Non-zero, directed back toward the bottom of the swing

C Equal to the velocity

D Non-zero, directed upward away from the ground

Why? One or two sentences. The words matter more than the option.

Explain your reasoning...

I chose this because...

The force(s) acting here are...

At that moment the velocity is... and the acceleration is...

The heavier/faster object...

How sure are you?

guessing fairly sure certain

0 words

Check my thinking →

Press Ctrl+Enter to submit

Your thinking map Q 1 / 8

Velocity-acceleration confusion

seen 1x · held with certainty ACTIVE

Session

1 answered

1 found

0 fixed

Finish & see report

Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

Transfer probe: same idea, new context (thrown ball → child on a swing). The target misconception is shown as a chip.

Re:Learn — Adaptive Multimodal Learning Environment · Project explainer

Page 7

Learn
Thinking map
Teacher
Evaluation
About

AIMAN

1 Answer
2 Diagnose
3 Fix
4 Verify
5 Track

CLEAR IN THIS CONTEXT ·
VELOCITY-ACCELERATION CONFUSION

Good — 1 of 2 contexts clear. One more, in a new setting.

Answer ✓ and no trace of "zero velocity means zero acceleration" in your words. · model 98%

IMPROVING ●●●

Next context: A spring →

Your thinking map
Q 1 / 8

Velocity-acceleration confusion

seen 1x · held with certainty

IMPROVING

Session

1 answered

1 found

0 fixed

Finish & see report

Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

Probe result: answer right and no trace of the misconception in the reasoning → “clear in this context”, 1 of 2. A second context follows; then a delayed recheck later in the session.

Re:Learn

LearnThinking mapTeacherEvaluationAboutAiman

1 Answer2 Diagnose3 Fix4 Verify5 Track

Does the model actually diagnose?

Held-out evaluation, split by question so every test item is unseen. Generated by `python ml/evaluate.py`.

Dataset: 1831 labelled rows over 45 items (85 hand-written). Split by item: 31 train / 14 held-out. Held-out items: P_FV_3, P_HF_1, P_IM_1, P_TL_1, P_VA_3, Q02, Q03, Q10, Q12, Q14, Q20, Q21, Q25, Q29.

METRIC	VALUE	TARGET	
Diagnosis macro-F1 (held-out questions)	0.989	≥ 0.8	✓ met
Confusable-pair accuracy (n=58)	0.966	≥ 0.75	✓ met
Flawed-reasoning recall (n=62)	0.984	≥ 0.7	✓ met
Unseen-misconception top-1 (leave-one-out)	0.854	≥ 0.6	✓ met
Calibration (ECE)	0.060	≤ 0.1	✓ met
Latency per diagnosis	3.4 ms	< 1000 ms	✓ met

Confusion matrix (held-out)

true ↓ / pred →	V=a	impetus	force→v	3rd law	heavier	none
V=a	69	0	0	0	0	0
impetus	0	82	0	1	0	0
force→v	0	0	78	0	0	0
3rd law	0	0	0	87	1	1
heavier	0	0	0	0	55	0
none	0	1	0	0	0	160

Does synthetic training data transfer to natural writing?

A model trained on *template rows only*, tested on all 85 hand-written rows: accuracy 0.933, macro-F1 0.932, abstained on 12%. Vague explanations routed to *unknown*: 94% (n=135); valid rows abstained: 0%. Flawed-reasoning false alarms: 0.6%.

Unseen misconceptions (leave one out of training; recover it from its description)

HELD OUT	N	TOP-1	TOP-1 WHEN ANSWERED	ABSTAINED	FALSE POSITIVES
Velocity-acceleration confusion	69	0.72	0.88	17%	1.5%
Impetus: motion carries a force	83	0.95	0.96	1%	0.9%
Force causes constant speed	78	0.81	0.94	14%	1.1%
Unequal action-reaction	90	0.91	0.98	7%	1.3%
Heavier objects fall faster	56	0.88	0.89	2%	1.7%

Per label

LABEL	P	R	F1	N
Velocity-acceleration confusion	1.00	1.00	1.00	69
Impetus: motion carries a force	0.99	0.99	0.99	83
Force causes constant speed	1.00	1.00	1.00	78
Unequal action-reaction	0.99	0.98	0.98	89
Heavier objects fall faster	0.98	1.00	0.99	55
No misconception	0.99	0.99	0.99	161

Decision rule: answer prior ×2.5 on mapped labels · abstain below 0.45 · unseen fallback when description similarity ≥ 0.1 with margin 0.02. Machine: x86_64 / CPython 3.13.16.

Your thinking mapQ 1 / 8

Velocity-acceleration confusionseen 1× · held with certaintyIMPROVING

Session

1answered

1found

0fixed

Finish & see report

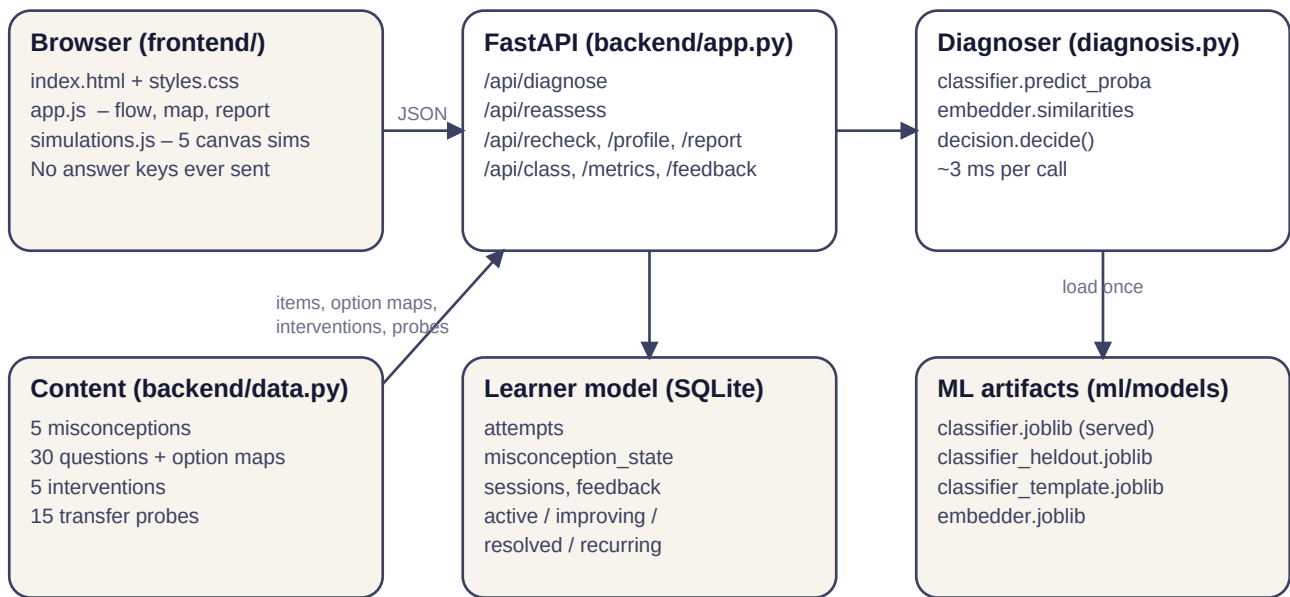
Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

The Evaluation tab serves `ml/reports/metrics.json` live: every PRD target, the confusion matrix, the unseen-misconception table and per-label scores.

Re:Learn — Adaptive Multimodal Learning Environment · Project explainer

Page 9

3. System architecture and data flow



Six parts, each in its own file. The browser never receives answer keys or option → misconception maps; all judging happens on the server.

One diagnosis, step by step

- **Browser** posts {learner, question_id, choice, explanation, confidence} to POST /api/diagnose.
- **app.py** looks the item up in data.py, checks correctness server-side and calls the Diagnoser.
- **Diagnoser** runs the classifier on the explanation (+ option text) → probabilities over the 5 misconceptions + NONE; runs the description embedder → similarity to every misconception description.
- **decision.decide()** applies the answer prior, the unseen-misconception fallback and the abstain threshold, then maps to a status.
- **Learner model** records the attempt; on a misconception it updates the state (active / recurring) and picks an unused transfer probe.
- **Response** carries status, label, confidence, probabilities, the misconception card, the intervention, the probe and the refreshed profile. The whole round trip is a few milliseconds.

One reassessment

POST /api/reassess diagnoses the probe answer the same way, then applies the resolution rule: **resolved = correct answer AND target misconception not detected in the reasoning**. A correct answer with too little reasoning is *inconclusive* (resolved = null) and the learner is asked for a sentence. Results drive the state machine in section 7.

4. Content layer (backend/data.py)

All physics knowledge lives in one module. Adding a misconception means one entry in each of four tables; the embedding fallback means it starts working from its description alone.

Misconception taxonomy (v1, from Force Concept Inventory research)

ID	Name	Learner believes	Confusable with	Simulation
VA_CONFUSION	Velocity–acceleration confusion	Zero velocity means zero acceleration	IMPETUS	Ball thrown up: v vs a arrows + v – t / a – t graphs
IMPETUS	Impetus: motion carries a force	A moving object carries the force that launched it, until it runs out	VA_CONFUSION, FORCE_VELOCITY	Projectile with free-body panel; ghost “throw force”
FORCE_VELOCITY	Force causes constant speed	A constant net force gives a constant speed	IMPETUS	Box under constant force: speedometer + v – t graph
THIRD_LAW	Unequal action–reaction	The heavier or faster object exerts the bigger force	HEAVIER_FASTER	Truck–car collision with equal force pair and force meters
HEAVIER_FASTER	Heavier objects fall faster	Heavier objects accelerate faster in free fall	THIRD_LAW	Bowling ball vs tennis ball in vacuum, with ruler and $a = F/m$ panel

Each entry also carries a **believes** paragraph, a **physics** paragraph and a learner-style **description** (“it stops so the acceleration is zero...”) that the embedding fallback compares explanations against.

Question bank

30 multiple-choice items across kinematics (6), Newton’s first law (7), second law (7), third law (6) and free fall (4). **Every wrong option is mapped to the misconception(s) that can produce it** — that mapping is the answer prior at inference time. Several options map to two misconceptions on purpose (the confusable pairs the PRD asks the model to tell apart).

```
{"id": "Q01", "topic": "kinematics", "concept": "accel_is_g_at_rest",
 "prompt": "A ball is thrown straight up. At the very top of its flight, what is its acceleration?",
 "options": [
   {"key": "A", "text": "Zero – it is momentarily at rest", "misconceptions": ["VA_CONFUSION", "IMPETUS"]},
   {"key": "B", "text": "9.8 m/s2 downward", "misconceptions": []},
   {"key": "C", "text": "9.8 m/s2 upward", "misconceptions": ["IMPETUS"]},
   {"key": "D", "text": "It depends on how hard it was thrown", "misconceptions": ["IMPETUS"]},
 "answer": "B",
 "ctx": {"obj": "the ball", "moment": "at the top", "agent": "the throw", "surface": "the ground"}}
```

concept tags the correct idea (used to generate sound-reasoning rows); ctx fills the slots of the dataset templates so explanations read naturally for each scenario.

Interventions (one per misconception)

- **headline** — the one-line reframing (“Force changes velocity. It does not set velocity.”)
- **explanation** — three short paragraphs: the mechanism, why the belief feels right, a test the learner can apply
- **worked_example** — numbered steps with numbers (“2 kg box, 4 N: $a = 2 \text{ m/s}^2$; after 3 s $v = 6 \text{ m/s}$ ”)
- **check_yourself** — a one-line self-test with the answer
- **simulation** — the key of the canvas animation to mount

Transfer probes

15 probes, three per misconception, each in a **different context** from the questions (thrown ball → swing → spring → bouncing ball; truck → mosquito → astronaut → book on a table). Probe options carry the same misconception maps, so a probe answer is diagnosed exactly like a question — which is what lets the system tell a clean pass from a lucky one.

5. The dataset (ml/generate_dataset.py)

1,831 labelled rows over **45 items** (30 questions + 15 probes), of which **85 are hand-written**. Each row is (item, chosen option, explanation) → label, where the label is one of the five misconceptions or NONE (sound reasoning).

How rows are generated — exactly the PRD's dataset plan

- **Hand-written seeds** (source = *hand*): 85 natural, varied explanations across the questions, including the PRD's A/R/S sentences. Reported separately in evaluation.
- **Student-style paraphrases** (source = *template*): 22 phrasings per misconception and 12 per correct concept, with slots ({obj}, {agent}, {moment}...) filled from each item's context, plus surface noise — typos, fillers ("i think", "cuz"), casing, dropped punctuation.
- **NONE rows**: correct option + sound reasoning for that item's concept.
- **Hard cases on purpose**: (a) the same wrong option with different misconceptions (confusable pairs); (b) the correct option with a misconceived explanation (flawed reasoning); (c) vague explanations ("idk", "just a guess") written to a separate `vague.csv` to test abstention.
- **Split by item, never by row** (`ml/data/split.json`): 31 train items / 14 held-out items, stratified by topic, one probe per misconception held out. Test questions are fully unseen.

Label	Rows
NONE	520
FORCE_VELOCITY	305
THIRD_LAW	298
IMPETUS	298
HEAVIER_FASTER	207
VA_CONFUSION	203

Row example from `responses.csv`:

```
row_id,item_id,item_kind,question,option_key,option_text,correct,explanation,label,source
R00003,Q10,question,"A cannonball is at the highest point of its arc...",B,"Gravity downward and a forward force from the cannon",0,the cannonball moves because it is carrying the forward momentum with it
```

Why synthetic? The PRD prescribes seeding by hand and expanding with generated paraphrases, and the risk register asks for hand-written rows to be reported separately — which the evaluator does (section 8). The feedback endpoint exports real learner corrections as new candidate rows, so the dataset can grow from classroom use.

6. The model and the decision rule

Classifier (ml/model.py)

Default backend: **TF-IDF + logistic regression**. Word 1–2-grams and character 2–5-grams of the explanation (weight 1.0) plus TF-IDF of the chosen option's text (weight 0.35), into a class-balanced logistic regression. Trains in under a second, predicts in ~3 ms on CPU. The question text is deliberately **not** an input, so the model cannot learn question-specific shortcuts.

Optional backend: **fine-tuned DeBERTa-v3-base** (ml/train_deberta.py, same item split). If ml/models/deberta/ exists and transformers is installed, the API loads it automatically.

Three trained models

File	Trained on	Used for
classifier.joblib	all 1,831 rows	served by the API
classifier_heldout.joblib	train-split items only	all held-out metrics
classifier_template.joblib	template rows only (no hand rows)	measuring transfer to natural writing

Description embedder (unseen-misconception fallback, FR-10)

Each misconception has a learner-style description. The embedder (TF-IDF cosine by default; sentence-transformers if installed) scores an explanation against every description. Its job is to catch misconceptions the classifier *cannot output* because they had no training rows — a new misconception works from its description alone.

Decision rule (ml/decision.py) — shared by the evaluator and the live API

```
probs = classifier(explanation, option_text)
probs = answer_prior(probs)          # ×2.5 for labels in the chosen option's map;
                                     # correct option: NONE ×1.3, item-level misconceptions ×1.6
if an UNSEEN label's description similarity leads the known ones
    (sim ≥ 0.1, margin ≥ 0.02, classifier conf < 0.97): label = unseen
if words < 3 or confidence < 0.45:    status = unknown
elif label == NONE and correct:       status = correct
elif label == NONE and wrong:        use strongest mapped misconception, else unknown
elif correct:                        status = flawed_reasoning
else:                                status = misconception
```

The unseen-fallback thresholds were tuned by leave-one-misconception-out grid search: 74 % true-positive rate on rows of the held-out misconception against 0.7 % false positives on rows of known ones. The item-level prior for correct answers exists because the option map of a correct option is empty — without it, flawed-reasoning detection would be systematically weakened.

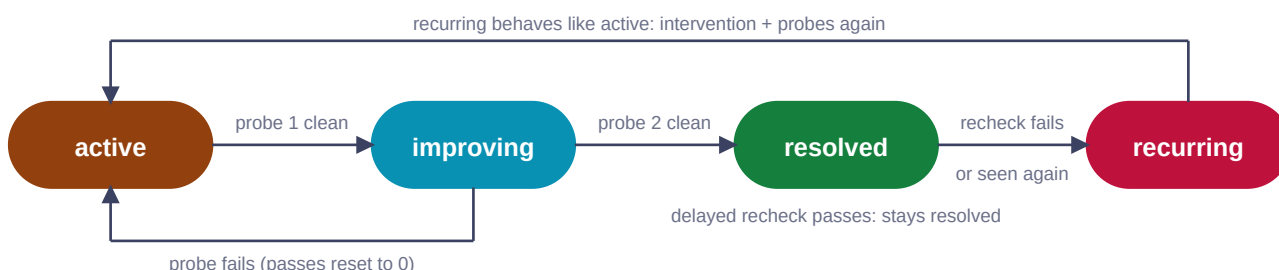
The four statuses

Status	Meaning	What the learner sees
correct	Right option, reasoning reads as sound	"Right answer, sound reasoning." → next question
flawed_reasoning	Right option, but a misconception is in the words	"You got it right, but the reasoning hides a misconception." → intervention + probe
misconception	Wrong option and the model can name why	Misconception card, confidence, intervention, probe

Status	Meaning	What the learner sees
unknown	Too little evidence (short/vague text or low confidence)	"I'd rather ask than guess" + sentence starters → resubmit

7. Resolution logic and the learner model (backend/learner_model.py)

A correct follow-up is never treated as proof of learning. A misconception is **resolved** only when the learner answers a transfer question correctly **and** the model finds no trace of the targeted misconception in the explanation — in **two different contexts**. A few attempts later a **delayed recheck** is injected; failing it re-opens the idea as **recurring**.



Rules

- **PROBES_REQUIRED** = 2 clean passes (env `RELEARN_PROBES_REQUIRED`); **MAX_PROBES** = 3 per misconception per learner.
- A failed probe resets the pass count to 0 and shows the intervention again, in belief mode, with another context.
- If all three probes are used with fewer than two passes, the idea is marked *improving* and settled by the delayed recheck instead.
- **RECHECK_GAP** = 2 attempts after resolution, `GET /api/recheck/{learner}` returns a probe; the frontend injects it before the next question.
- Recheck pass → stays resolved (“recheck passed” in the report). Recheck fail → *recurring*, `detected_count` +1, intervention again.
- A different misconception surfacing in a probe's reasoning is added to the map as a new active idea.
- Self-rated confidence “certain” at detection sets **confidently_held** → “held with certainty” badge (FR-12).
- Inconclusive probe (right answer, too little text) → resolved = null; the learner is asked for one more sentence.

Storage (SQLite, one local file)

Table	Columns (abridged)
attempts	learner, session_id, ts, phase (question probe recheck), item_id, choice, correct, explanation, self_confidence, status, label, model_confidence, target, resolved, latency_ms
misconception_state	learner, misconception, state, detected_count, resolved_count, probes_passed, probes_failed, probes_used, confidently_held, first/last_detected, resolved_at, recheck_due_after, recheck_done

Table	Columns (abridged)
sessions	learner, started, finished, question_order
feedback	learner, attempt_id, suggested_label, note, ts — exported as training rows

The profile (answered, right first time, flawed reasoning, per-misconception states, timeline) is recomputed from these tables on every request, so nothing can drift. An “unknown → elaborate” resubmission counts as the same question, not a new one.

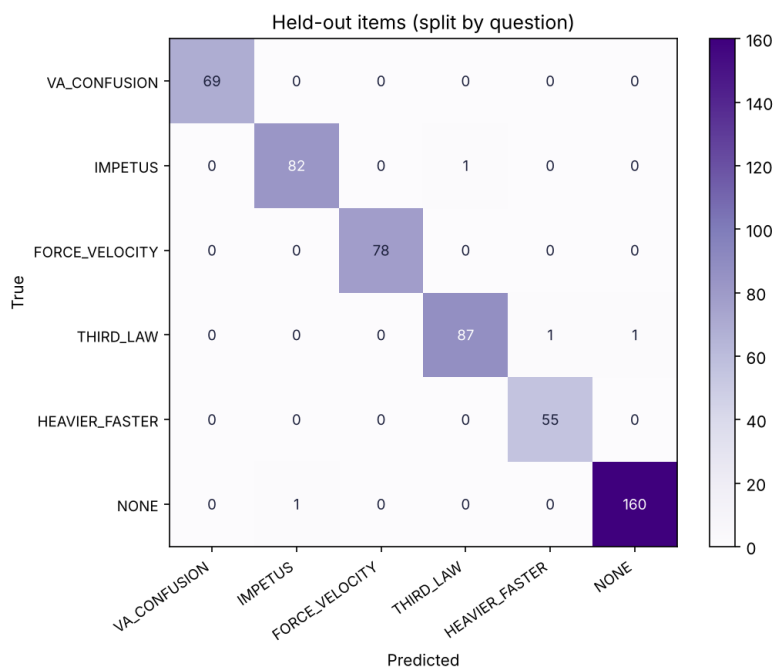
8. Evaluation: does it actually diagnose? (ml/evaluate.py)

Everything below is produced by python `ml/evaluate.py`, written to `ml/reports/` and served live at `/api/metrics`. All held-out numbers use the model trained on train-split items only.

Metric (PRD §10)	Definition	Value	Target	
Diagnosis macro-F1	held-out questions, n=363	0.989	≥ 0.80	met
Confusable-pair accuracy	chosen option maps to 2+ misconceptions, n=58	0.966	≥ 0.75	met
Flawed-reasoning recall	right-answer-wrong-reason rows flagged, n=62	0.984	≥ 0.70	met
Unseen-misconception top-1	leave-one-misconception-out, embedding fallback	0.854	≥ 0.60	met
Calibration (ECE)	10-bin expected calibration error of shown confidence	0.060	≤ 0.10	met
Latency	per diagnosis, this CPU	3.3 ms	< 1 s	met

Secondary checks

- **Transfer to natural writing:** a model trained on template rows only, tested on all 85 hand-written rows: accuracy 0.933, macro-F1 0.932 (abstained on 12%). This is the most honest single number in the project.
- **Abstention:** 94% of vague explanations (n=135) are routed to *unknown*; 0% of valid rows are wrongly abstained.
- **Flawed-reasoning false alarms:** 0.6% of correct-answer-sound-reasoning rows flagged.
- **Unseen-misconception false positives:** 1.3% of known-label rows routed to the held-out label.



Confusion matrix on held-out items (true rows, predicted columns).

Unseen misconceptions, one at a time

Held out of training	n	top-1	top-1 when answered	abstained	false positives
VA_CONFUSION	69	0.72	0.88	17%	1.5%
IMPETUS	83	0.95	0.96	1%	0.9%
FORCE_VELOCITY	78	0.81	0.94	14%	1.1%
THIRD_LAW	90	0.91	0.98	7%	1.3%
HEAVIER_FASTER	56	0.88	0.89	2%	1.7%

Honesty note. The headline numbers are high partly because the data is synthetic — templated paraphrases are easier than real classroom writing. The project reports the hand-written transfer number separately for exactly that reason, and the feedback export exists so real corrections can be folded into the next training run.

9. API reference (backend/app.py)

FastAPI with interactive docs at /docs. All payloads are JSON.

Endpoint	Body / params	Returns
GET /api/health	—	model backend, labels, probes_required, recheck_gap
GET /api/questions	—	30 questions without answer keys, sentence starters
GET /api/misconceptions	—	taxonomy: name, short, believes, physics, colour, confusable_with
POST /api/session/start	learner, mode (quick topic full), topic?	session_id, question_order, questions, profile
POST /api/diagnose	learner, question_id, choice, explanation, confidence	status, label, confidence, probabilities, notes, misconception card, intervention, probe, state, profile
POST /api/reassess	learner, misconception, probe_id, choice, explanation, confidence, phase (probe recheck)	resolved (true / false / null), target_present, state, probes_passed, next probe or intervention, new_misconception?
GET /api/recheck/{learner}	—	a recheck probe if one is due, else null
GET /api/profile/{learner}	—	learner model: counts, per-misconception states, timeline
GET /api/report/{learner}	—	session summary (right first time, found, fixed, improving, still open, recurring...) + details
POST /api/session/finish	session_id	ok
POST /api/feedback	learner, attempt_id, suggested_label, note	stored; GET /api/feedback/export returns rows joined with the attempt
GET /api/class	—	teacher view: learners, misconception frequency by state, status counts
GET /api/metrics	—	ml/reports/metrics.json (or a hint to run evaluate.py)
DELETE /api/learner/{learner}	—	reset a learner (demo convenience)

Example

POST /api/diagnose

```
{"learner": "Sam", "question_id": "Q01", "choice": "B",  
  "explanation": "It is still slowing down because of the throw force.", "confidence": "fairly_sure"}
```

```
→ {"status": "flawed_reasoning", "correct": true, "label": "IMPETUS", "confidence": 0.992, "latency_ms": 123,  
  "probabilities": {"IMPETUS": 0.992, "FORCE_VELOCITY": 0.004, ...},  
  "misconception": {"name": "Impetus: motion carries a force", "believes": "...", "physics": "..."},  
  "intervention": {"headline": "A force acts only while something is pushing...", ...},  
  "probe": {"id": "P_IM_1", "context": "A bowling ball on ice", "prompt": "...", "options": [...]},  
  "state": "active", "profile": {...}}
```

10. The frontend and the simulations

A single page, no build step: `index.html`, `styles.css`, `app.js`, `simulations.js`. Five tabs: **Learn** (the loop), **Thinking map**, **Teacher**, **Evaluation**, **About**. A persistent sidebar shows the live thinking map and session counters; a progress rail shows Answer → Diagnose → Fix → Verify → Track.

app.js — the flow

- `showStart` → `showQuestion` → `submitDiagnose` → `showVerdict` (branches by status) → `showProbe` → `submitReassess` → `showProbeResult` → `advance`.
- `advance()` asks `/api/recheck` first; if a recheck is due it is shown before the next question with a “Quick recheck” banner.
- *unknown* re-opens the same form with the learner's text pre-filled so they can add to it, not retype.
- Report: tiles, per-idea rows with a probe ribbon, a status timeline, JSON download.
- Teacher: stacked bars per misconception (recurring / active / improving / resolved) across all learners on the machine.
- Evaluation: renders `metrics.json` — target table with met/missed, confusion matrix grid, unseen table, per-label scores.

simulations.js — five canvas animations

Each simulation is a pure `draw(ctx, W, H, t, mode)` over a looping time t ; a small controller handles DPR scaling, play / pause / replay / step, a scrubber, and the **belief** ↔ **physics** mode. In belief mode a red dashed ghost shows what the misconception predicts next to what actually happens — the learner sees the contradiction rather than being told about it.

Key	Scene	What the physics mode shows	What the belief mode adds
throw_up	Ball thrown straight up	v arrow shrinks to 0 and flips; a arrow constant; v - t and a - t graphs	a arrow shrinking to 0 at the top; dashed dip in a - t
forces_in_flight	Projectile after release	gravity only; free-body panel; “inertia keeps it moving”	a fading “throw force” arrow with “no object applies this”
constant_force	Box on frictionless track	speedometer rising; v - t line rising; $F = m \cdot a$	dashed flat “constant speed” line
collision	4000 kg truck hits 1000 kg car	equal 40 kN force pair; car accelerates 4× more	unequal ghost arrows; meters read 60? / 15?
free_fall	7 kg vs 0.06 kg in vacuum	both fall together; $a = F/m$ panel; 5 m ruler	ghost of the heavy ball landing first

Accessibility and non-functional requirements

- Full keyboard flow: options are real radio buttons, Ctrl+Enter submits, visible focus rings everywhere.
- `prefers-reduced-motion`: simulations do not autoplay; Step and the scrubber replace animation.
- Phone width: single column, sidebar moves below, tabs scroll horizontally.
- Transparency: confidence and top candidates always shown; abstains on weak evidence; “that’s not what I meant” on every diagnosis.

- Privacy: answer keys and option maps never reach the browser (covered by a test); learner data stays in a local SQLite file.

11. File-by-file guide

Path	What it does
run.py	One-command launcher: generates data, trains and evaluates if artifacts are missing, starts uvicorn, opens the browser. Flags: --retrain, --no-open, --port.
start.bat / start.sh	Create a venv, install requirements, run run.py.
requirements.txt	fastapi, uvicorn, scikit-learn, pandas, numpy, joblib, matplotlib, pytest, httpx. Optional (commented): torch, transformers, sentence-transformers.
.vscode/	launch.json (F5: run everything / API only / ML steps / tests), tasks.json (install, generate, train, evaluate, test, start), settings.json, extensions.json.
backend/app.py	FastAPI routes, session logic, probe selection, response assembly.
backend/data.py	Taxonomy, 30 questions with option maps, interventions, 15 probes, helpers (public_item strips keys).
backend/diagnosis.py	Diagnoser: loads classifier + embedder once; diagnose(item, choice, explanation) → Decision.
backend/learner_model.py	SQLite store: attempts, states, probes, recheck scheduling, profile, class view, feedback.
ml/generate_dataset.py	Template banks, hand-written seeds, noise, hard cases → responses.csv, vague.csv.
ml/model.py	MisconceptionClassifier (TF-IDF + LogReg), DebertaClassifier (optional), DescriptionEmbedder.
ml/decision.py	DecisionConfig + decide(): answer prior → unseen fallback → abstain → status.
ml/train.py	Item-level split; trains served, held-out and template-only models + embedder.
ml/evaluate.py	All metrics → metrics.json, report.md, confusion_matrix.png.
ml/train_deberta.py	Optional transformer backend on the same split.
ml/data/	responses.csv (1,831 rows), vague.csv (135 rows), split.json.
ml/models/	classifier.joblib, classifier_heldout.joblib, classifier_template.joblib, embedder.joblib.
ml/reports/	metrics.json, report.md, confusion_matrix.png.
frontend/	index.html, styles.css, app.js, simulations.js.
tests/test_api.py	13 end-to-end tests: four statuses, A/R/S, resolution, recurrence, inconclusive probe, privacy, feedback, metrics.
demo/DEMO_SCRIPT.md	Three-minute judge walkthrough with exact inputs.
docs/screenshots/	The images used in this document.

12. How to run (Windows / VS Code)

Fastest: double-click

Open the project folder in File Explorer and double-click **start.bat**. It creates a virtual environment, installs the packages (1–3 minutes the first time), trains the model (~3 s) and opens `http://127.0.0.1:8000`. Keep the black window open; close it to stop.

VS Code

- **File** → **Open Folder** → select the folder that contains `run.py` (not a parent folder, not the zip).
- **Terminal** → **New Terminal**, then:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python run.py
```

- Or press **F5** — the launch configuration “Re:Learn — run everything” is preselected.
- If PowerShell refuses to activate: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or use `.venv\Scripts\activate.bat`.
- If python is not recognised, use `py`.

Other commands

Command	Effect
<code>python ml/generate_dataset.py</code>	regenerate <code>ml/data/responses.csv</code> and <code>vague.csv</code>
<code>python ml/train.py</code>	retrain the three models + embedder (~3 s)
<code>python ml/evaluate.py</code>	full evaluation → <code>ml/reports/</code>
<code>python -m pytest -q tests</code>	run the 13 tests
<code>python run.py --retrain</code>	everything from scratch
<code>uvicorn backend.app:app --reload</code>	API only, auto-reload while editing
<code>python ml/train_deberta.py --epochs 4</code>	optional DeBERTa backend (needs torch + transformers)

Environment knobs

Variable	Default	Meaning
<code>RELEARN_PROBES_REQUIRED</code>	2	clean transfer probes needed to resolve
<code>RELEARN_RECHECK_GAP</code>	2	attempts to wait before the delayed recheck
<code>RELEARN_DB</code>	<code>backend/relearn.db</code>	SQLite file
<code>RELEARN_BACKEND</code>	auto	auto (DeBERTa if present) or sklearn
<code>RELEARN_EMBEDDER</code>	tfidf	tfidf or st (sentence-transformers)

13. Three-minute demo script

Full version with every input in demo/DEMO_SCRIPT.md; every input is also asserted in the tests.

#	Moment	Input	What judges see
1	Same wrong answer, two diagnoses	Q1, pick A, "It stops at the top, so velocity is zero." (certain) — then as a second learner: Q1, pick A, "The force of the throw has run out."	Velocity–acceleration confusion vs Impetus; "held with certainty" badge
2	Right answer, wrong idea	Q1, pick B (correct), "It is still slowing down because of the throw force."	flawed_reasoning → Impetus — an LMS would mark this fully correct
3	The fix	scroll; tick "Show what the belief predicts"	belief ghost vs physics in the animation; worked example
4	Verify, twice	probe 1: B, "velocity is zero for an instant but gravity still acts so the acceleration is not zero"; probe 2: B, "net force is not zero at the lowest point so there is a large acceleration even though $v = 0$ "	improving 1/2 → resolved 2/2
5	Ask, don't guess	Q3, pick A, "idk" → then add "the bowling ball is heavier so gravity pulls it harder and it falls faster"	unknown → asks for a sentence → Heavier objects fall faster
6	Delayed recheck	answer the injected recheck wrongly: A, "it is not moving so no acceleration"	"It came back" → recurring
7	Evidence	Finish & see report; Teacher tab; Evaluation tab	report tiles and timeline; class bars; all PRD targets met

Closing line: "Diagnose the idea, not the answer. Fix the idea, not the score. And never call it learned until it survives a new context and a delay."

14. PRD traceability

ID	Requirement	Where	Status
FR-1	Labelled misconception dataset covering correct, incorrect and right-answer-wrong-reason rows	ml/generate_dataset.py	Done — 1,831 rows
FR-2	Trained classifier returning label + confidence	ml/model.py, ml/train.py	Done (TF-IDF/LogReg; DeBERTa optional)
FR-3	Differentiation via option → misconception maps	backend/data.py, ml/decision.py	Done
FR-4	Flawed-reasoning detection	ml/decision.py	Done — recall 0.98
FR-5	Adaptive intervention: explanation, worked example, simulation	data.py, simulations.js	Done
FR-6	Resolution = correct answer AND target absent	backend/app.py reassess	Done
FR-7	2–3 probes in different contexts + delayed recheck	learner_model.py	Done — 2 probes, 3 contexts, recheck
FR-8	Learner model: active / recurring / resolved	learner_model.py	Done (+ improving)
FR-9	Unknown handling: ask instead of guessing	ml/decision.py, app.js	Done
FR-10	Unseen-misconception detection via description similarity	ml/model.py DescriptionEmbedder	Done — top-1 0.85

ID	Requirement	Where	Status
FR-11	Evaluation suite	ml/evaluate.py, Evaluation tab	Done
FR-12	Confidence capture → confidently held	UI + learner model	Done
FR-13	Session report and progress indicators	app.js	Done
FR-14	Teacher class view	/api/class, Teacher tab	Done
§15	“That’s not what I meant” feeding training data	/api/feedback, /api/feedback/export	Done
M7	Scripted demo covering all four statuses + resolved / not-resolved	demo/, tests/	Done

Non-functional requirements

NFR	How it is met
Latency < 1 s on a laptop CPU	~3 ms per diagnosis
Privacy	answer keys and option maps never sent to the browser (tested); SQLite stays local
Accessibility	keyboard flow, focus states, reduced-motion mode, phone width
Transparency	confidence always shown; abstain threshold; learner can dispute
Extensibility	a new misconception needs a description, an intervention and a probe; works before retraining via the embedding fallback

15. Limitations and what comes next

Limitations to state openly

- **Synthetic data.** Held-out scores are inflated by templated paraphrases. The hand-written transfer number (0.93) is the realistic one, and real classroom text will be harder — negation (“zero velocity does *not* mean zero acceleration”) is the known weak spot of a bag-of-n-grams model.
- **Five misconceptions, one domain.** Mechanics only, text input only, per the PRD’s non-goals.
- **Delayed recheck is within-session.** The PRD’s open question (same session vs next session) is answered with “same session” for v1; the data model already stores enough to do cross-session rechecks.
- **No accounts.** A learner is a name; the teacher view aggregates whatever is in the local database.

Roadmap

- Collect real explanations through the app; label with the feedback export; retrain — the pipeline is one command.
- Switch to the DeBERTa backend once real data exists (script included), which handles negation and paraphrase far better.
- Cross-session recheck and spaced re-testing of resolved ideas.
- A structured reasoning picker as a fallback for learners who write very little (PRD §15).
- Second domain (algebra or programming) to show the approach generalises: the content layer is domain-agnostic — items, option maps, interventions, probes.

Everything in this document is reproducible with `python run.py --retrain` and `python -m pytest tests`.

16. Version 2: beyond the PRD

Everything in sections 1–15 is still true. Version 2 adds the capabilities below, each built against a line in the hackathon brief (“answers, working, or code”, “generate or select an intervention”, “demonstrated understanding across attempts”, “misconceptions not seen during training”).

Three input modalities

- **Choice + explanation** — the original mode (30 questions).
- **Show your working** — five numerical items. The learner types a number *and* the working; explainable rules read the working (“ $v = F/m$ ” called a speed $\rightarrow$ force $\rightarrow$ velocity; “ $a = m \cdot g$ ” $\rightarrow$ heavier-falls-faster; force scaled by the mass ratio $\rightarrow$ unequal action–reaction). A right number with a flawed working is still flagged.
- **Draw the forces** — four free-body-diagram items. The learner places arrows (force, direction, size) on the object. The diagram is diagnosed directly: a ‘throw force’ after release, a forward push on a coasting puck, an engine arrow larger than drag at steady speed, or no forces at the top of a flight each map to a misconception. When a drawing could mean two ideas, the learner’s sentence decides. The correct diagram is drawn next to theirs.

The screenshot shows the Re:Learn interface for a physics problem. The question is: "A hockey puck slides across frictionless ice at constant velocity (to the right). Draw all the forces on it." The diagram editor shows a puck on a horizontal line labeled "frictionless ice". Three arrows are drawn: a blue arrow pointing up labeled "normal force", an orange arrow pointing down labeled "gravity", and a red arrow pointing right labeled "forward push". A green dashed arrow labeled "motion" points right. To the right of the diagram is a control palette with three sections: "1 · Force" with buttons for "gravity", "normal force", "forward push", and "friction"; "2 · Direction" with buttons for "up", "down", "left", and "right" (selected); and "3 · Size" with buttons for "small", "medium" (selected), and "large". Below the palette is an "Add arrow" button and a list of three items: "gravity · down · medium", "normal force · up · medium", and "forward push · right · medium", each with a red 'X' icon. Below the diagram is a text box for "Why did you draw it that way?" with the text "the hit is still pushing it along the ice". Below the text box are buttons for "Say it" and "Photo of handwriting". Below these are three buttons: "I chose this because...", "The force(s) acting here are...", and "At that moment the velocity is... and the acceleration is...". Below these are two buttons: "The heavier/faster object..." and "How sure are you?". The "How sure are you?" section has buttons for "guessing", "fairly sure" (selected), and "certain". Below this is a "Diagnose my thinking" button. On the right side of the interface is a "Your thinking map" section with a "Velocity-acceleration confusion" entry, a "Session" section with "3 answered", "1 found", and "1 fixed" items, and a "Finish & see report" button. At the bottom right, there is a model information section: "Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence".

The free-body-diagram editor: palette $\rightarrow$ direction $\rightarrow$ size $\rightarrow$ add. The arrows, not just the answer, are what gets diagnosed.

MISCONCEPTION FOUND · ANSWER ×

The answer is wrong — and I can see why.

model confidence 90% · 0.4 ms · you said: fairly sure · from your working

In your working: The working scales the force by the mass ratio. Masses change the accelerations ($a = F/m$), never the pair of forces, which are always equal.

Correct answer: 40000 N

1. Newton's third law: $F(\text{car on truck}) = F(\text{truck on car}) = 40\,000\text{ N}$

2. Accelerations differ: $\text{car } 40\,000/1000 = 40\text{ m/s}^2$, $\text{truck } 40\,000/4000 = 10\text{ m/s}^2$

Unequal action-reaction

The heavier or faster object exerts the bigger force

WHAT YOUR REASONING SUGGESTS

In any interaction the bigger, heavier, faster or 'more active' object pushes harder than the other one. The winner of a collision exerted the bigger force.

How sure is the model?

90% that this is the idea behind your working

working pattern matched: The working scales the force by the mass ratio. Masses change the accelerations ($a = F/m$), never the pair of forces, which are always equal.

► That's not what I meant

Your thinking map Q 6 / 9

Velocity-acceleration confusion

seen 1× · held with certainty

RESOLVED

Impetus: motion carries a force

seen 1×

RESOLVED

Unequal action-reaction

seen 1×

ACTIVE

Session

6	3	2
answered	found	fixed

Finish & see report

Model: tfidf-logreg · 2 probes to resolve · confidence always shown · abstains on weak evidence

Working mode: the rule that caught the error is quoted back, then the correct working.

Voice and photo

A “Say it” button dictates the explanation (browser speech recognition, Chrome/Edge). A “Photo of handwriting” button accepts a photo of handwritten working: the server stores it, an LLM transcribes it when an API key is configured, otherwise Tesseract.js reads it in the browser; the learner corrects the transcript before submitting, and the teacher sees the image when labelling.

Personalised fixes — “generate or select”

Every intervention now opens with the learner's own sentence. The phrase that carries the misconception is highlighted and answered directly (“You said ‘...the force of the throw has **run out**’. A force cannot run out because it was never stored in the object...”). This is rule-based and always on; with ANTHROPIC_API_KEY set the bridge is generated by an LLM from the learner's exact words, the question and the physics — so the system now *generates* interventions as well as selecting them, with a safe offline fallback.

WRITTEN FOR YOU FROM YOUR WORDS

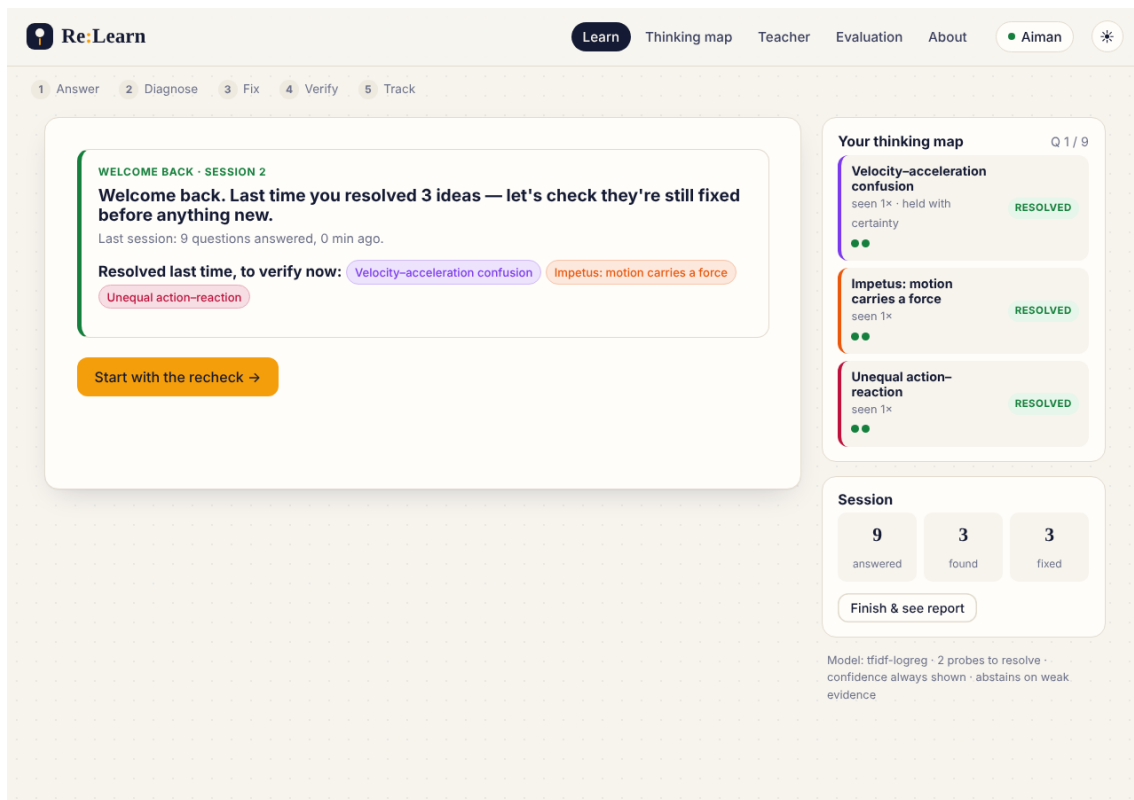
You said: “the hit is **still** pushing it along the ice”

The key phrase is “still”. Nothing travels inside the object. Draw the forces a moment after release: gravity is the only arrow you can justify.

“Written for you”: the learner's words, the trigger phrase highlighted, a reply to that phrase.

Cross-session memory

A returning learner is greeted with what was resolved last time and is re-checked on it in a new context *before* anything new. Passing earns “held across sessions” — the strongest evidence of learning the system collects, and the direct answer to the brief's “demonstrated understanding across attempts”. A struggle signal (time to answer, number of rewrites) is recorded too; a hesitant “certain” is not counted as a confidently held belief.



Session 2: three ideas resolved last time are verified first.

From synthetic data to real data

- **Label real responses** (Teacher tab): every real explanation typed into the app is listed with the model's label; the teacher confirms or corrects it.
- **Export** writes `ml/data/real_responses.csv`; `python ml/evaluate.py` then reports accuracy on real rows separately, and `python ml/train.py --real` folds them into the served model.
- **Discover**: explanations the model abstained on are clustered (TF-IDF + k-means); a cluster that matches no known description is surfaced as a candidate *new* misconception with its top terms and example sentences. Add a description to `data.py` and the embedding fallback diagnoses it immediately.

Voice explanation with live NLP, and a small voice chatbot

The explanation panel under every question is voice-first: the learner taps the mic and speaks; the transcript appears and can be edited (typing and a handwriting photo still work). A live NLP readout shows what was heard — concepts, reason clause, tone — and nudges for more when needed. The sidebar holds a small “Ask the tutor” voice chatbot: a spoken physics question is answered from a knowledge base and read aloud; quiz answers spoken there are redirected to the explanation panel.

Tutor reply: NLP on the explanation box

After every submit the learner receives a reply written about *their* sentence, claim by claim — the brief’s “answer based on the student's understanding”. `backend/claims.py` holds a bank of about seventy claim patterns students actually write, each with a verdict (true / partly / false) and a reply to that specific claim; matching is item-aware, so the item's own misconceptions are read first. `backend/tutor_response.py` then walks the learner's claims in the order they said them — *You said “the ball stops” — true. Then you said “velocity is zero” — correct for that instant. You also said*

“acceleration must be zero” — *this is the step that breaks...* — followed by one line of physics and a question to carry forward. If no false claim was spoken aloud, the hidden step is named from the giveaway phrase; correct answers get credit for the specific true claims; vague explanations are asked for reasoning rather than given a generic verdict. Rule-based and offline by default; with OPENAI_API_KEY or ANTHROPIC_API_KEY set (no extra packages), backend/llm.py generates a GPT-style reply grounded in the same claim analysis, falling back to the rules on any error. The explanation can be typed, dictated or photographed.



TUTOR NLP

Aiman, a very common way to think about this, and it is wrong in one specific, fixable place.

You said “the ball stops” — yes, it is momentarily at rest — true. Then you said “velocity is zero” — the velocity really is zero for that instant — that part is correct. You also said “acceleration must be zero” — this is the step that breaks: acceleration is not 'how fast it moves' but 'how fast the velocity is changing'. At that instant the velocity is changing from upward to downward — so the acceleration is a full 9.8 m/s^2 downward even though the speed is zero.

In one line: velocity and acceleration are different quantities — v can be zero while a is not. Is the velocity about to change at that instant? If yes, the acceleration cannot be zero.

You sounded certain, which is worth noticing — this idea feels obviously true, and that is exactly why it survives.

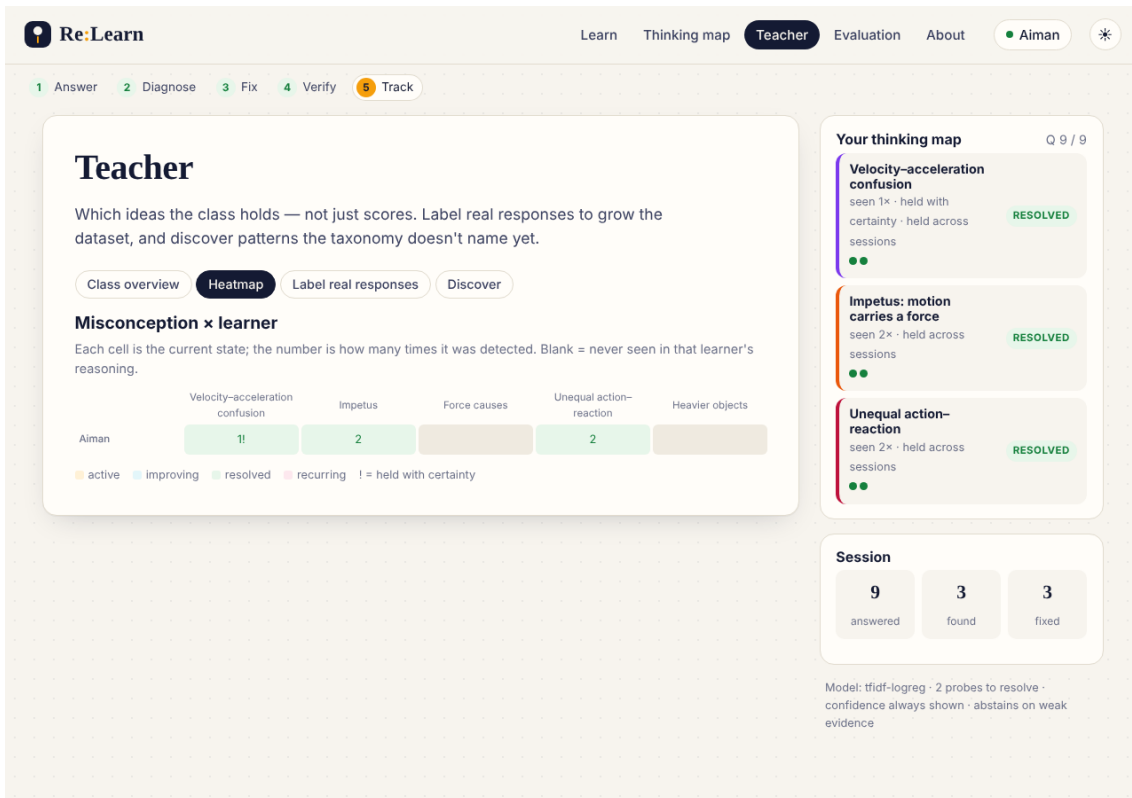
Below: the explanation, an animation where you can switch on “what the belief predicts”, and a worked example. Then a different question on the same idea.

velocity acceleration rest

The tutor reply at the top of every result: the learner's phrase quoted, credit for what was right, the physics, a question.

Teacher and evaluation

- Misconception × learner heatmap; confidently-held wrong ideas; which questions expose which ideas.
- Calibration (reliability) diagram on the Evaluation tab; real-row accuracy once labelled.
- Per-learner PDF report; dark mode; 24 end-to-end tests.



Teacher heatmap: state and detection count per learner and misconception (! = held with certainty).